S-X8 v4.3: A 7.50-Bits-Per-Weight Quantization Format with FP16-Level Quality and Portable Decoding

Martí Vidal Leandro

MarlaLabs — Independent AI Lab · marlalabs.com

Abstract. This paper presents S-X8 v4.3, a weight quantization format for large language models that stores **7.50 bits per weight** — over 11% fewer bits than the widely used Q8_0 (8.50 bpp) — while delivering **FP16-level quality** (perplexity within +0.2% of the unquantized model on wikitext-2, inside statistical noise) and **9× less perplexity loss than Q8_0** (+0.26% vs. +2.40% over FP16). On Qwen3.5-4B, the format reduces the text payload by 11.6% and the complete model (including vision and multi-token prediction towers) by 15% compared to a GGUF Q8_0 + mmproj deployment, and runs decoding **faster than Q8_0 in real-world use** (63.8 vs. 40–54 tokens/s on an RTX 5060 Ti). The format is byte-aligned, uses only ~9–10 ALU operations per weight for decoding (no shared memory, no shuffles, no tensor-core dependency), making it portable across GPU architectures and backends. Quality is validated with a strict protocol (perplexity, Winogrande_s, HellaSwag, ARC-Challenge, MMLU) against FP16 and Q8_0 on the same machine, and the format is integrated as a native type in a llama.cpp fork, including a matrix-multiply kernel for prompt processing. Honest limitations are reported: on multiple-choice benchmarks all three formats are statistically indistinguishable, and prompt throughput in llama.cpp remains below Q8_0.

Keywords: quantization · LLM compression · perplexity · llama.cpp · portable kernels

1 Introduction

Large language models are memory-bound during inference: generation speed is dominated by reading the weight matrix from VRAM on every token. Reducing the bytes per weight (bpp) therefore directly improves speed and capacity. The de-facto standard 8-bit format, GGUF **Q8_0** (8.50 bpp), offers good quality but is byte-wasteful: its block structure and symmetric scaling consume bits that carry little information.

This work introduces **S-X8 v4.3**, a format that quantizes weights in blocks of 32 using *four adaptive range strategies* per 8-weight sub-block (chosen per sub-block by mean squared error), a 6-bit hierarchical level encoding (4+2 bits), exact FP16 scales, an explicit configuration byte, and a 1-byte PCA correction coefficient per block. This yields exactly **30 bytes per block = 7.50 bpp** — the true, fully accounted bit count — with quality essentially equal to the unquantized model. S-X denotes the family of formats built on this methodology (adaptive range strategies, hierarchical levels, output-side PCA correction); S-X8 v4.3 is its validated 8-bit member, and the methodology was conceived and developed by the author (Martí Vidal Leandro, 2025–2026).

The contributions of this work are:

1. A complete byte-level format specification (Section 2) whose decoder is *portable by design*: ~9–10 ALU ops per weight, byte-aligned, no shared memory, no shuffles, no special instructions.
2. A rigorous evaluation protocol on Qwen3.5-4B (Section 3) comparing FP16, S-X8 v4.3 and Q8_0 on perplexity and five multiple-choice benchmarks, all measured on the same machine (Section 4).
3. Native integration in a llama.cpp fork as type `GGML_TYPE_SX8`, with both a vector-dot decode kernel and a tensor-core matrix-multiply (MMQ) kernel for prompt processing (Section 5).
4. An honest discussion of where the format wins and where it does not (Section 7).

2 The S-X8 v4.3 Format

2.1 Block layout (30 bytes, 7.50 bpp)

A block covers **32 consecutive weights** of one row of a weight matrix. The 30-byte layout is:

Field	Size	Description
`dmin`, `dmax`	2+2 B	FP16 exact range endpoints (10-bit mantissa, lossless vs. FP32)
`config`	1 B	4 × 2 bits: range strategy per 8-weight sub-block
`qh`	16 B	High nibbles of 6-bit levels (4 bits × 32 weights)
`ql`	8 B	Low 2 bits of 6-bit levels (2 bits × 32 weights)
`coeff`	1 B	PCA correction coefficient (2 signed 4-bit bases)
Total	**30 B**	**= 7.50 bpp, byte-aligned**

Table 1: S-X8 v4.3 block layout. Every byte is accounted for; the bit count is exact.

2.2 Adaptive range strategies

Each 8-weight sub-block selects, out of 4 strategies, the range that minimizes the sub-block MSE: (i) full range *[dmin, dmax]*, (ii) lower quarter *[dmin, dmin+q]*, (iii) upper quarter *[dmax−q, dmax]*, and (iv) central half *[dmin+q, dmax−q]*, where *q=(dmax−dmin)/4*. The chosen strategy is stored in 2 bits of the configuration byte. It was verified empirically that additional candidate ranges (including inverted ranges) add no measurable quality (+0.00%), so the 4-strategy set is optimal for 2 bits of metadata.

2.3 Levels and PCA correction

Each weight is encoded as a 6-bit level *lv = (hi << 2) | lo* with the high 4 bits in `qh` and the low 2 bits in `ql` — a hierarchical split derived from the two-level structure observed in the motivating analysis (Appendix A). The decoder computes *rlo, rhi* branchlessly from the strategy, then *step = (rhi−rlo)·0.015873* and *w = rlo + step·lv*.

A **PCA correction** refines the reconstruction: per block-of-K the format stores 2 basis vectors b_0, b_1 (shared across output columns) and per (block, column) one signed coefficient byte (c_0, c_1): $w \mathrel{+}= c_0 \cdot s_0 \cdot b_0[t] + c_1 \cdot s_1 \cdot b_1[t]$. Using the linearity of the dot product, the correction is applied to the *output* of a matrix multiply as two scalar accumulators per block of K (Z_0, Z_1, computed once per layer per token), turning the per-weight PCA cost (2 FMA + 2 FP32 loads per weight) into 2 FMAs per block (≈0.06 per weight). All kernels in this work use this reformulation.

2.4 Portable decoding

Decoding S-X8 requires no shared memory, no warp shuffles, no texture units and no tensor cores: only byte extraction, a branchless strategy selection and two FMAs per weight. The same kernel therefore runs unchanged on architectures separated by ten years (validated on an RTX 5060 Ti, Blackwell SM120, and a GTX 960M, Maxwell). Porting to ROCm, Metal or SIMD CPU backends is straightforward.

2.5 Container format (.sx8v43)

Models are packaged in a self-contained byte-aligned container (`.sx8v43`): a magic header (`SX43FILE` , version 1), the model metadata, and one record per tensor holding its 30-byte blocks plus the PCA bases and scales. The container is pickle-free and verified byte-exact round-trip (381/381 tensors; PPL from file identical to in-memory). Vision and MTP

towers are quantized inside the same file — the complete model in a single artifact, unlike GGUF which requires a separate FP16 mmproj. Full layout in `SX8_FLASH_V4_3_CONTAINER.md`.

3 Evaluation Methodology

3.1 Model and hardware

Evaluation is performed on **Qwen3.5-4B**, a hybrid 2026 model: 24 Gated-DeltaNet layers (linear attention) and 8 full GQA attention layers, with a vision tower and a multi-token-prediction head (4.66B parameters in total, of which the 4.2B text transformer is fully quantized; norms, conv1d, vision and MTP are kept in FP16, mirroring the reference GGUF which stores vision in a separate mmproj file). All measurements run on a single **RTX 5060 Ti 16 GB** (SM120, 448 GB/s) with CUDA 13.0, PyTorch 2.11 and llama.cpp (commit 7c203670f).

3.2 Protocol

- **Perplexity** on wikitext-2 test (297,053 tokens), 512-token windows with 128 stride (standard community protocol), fresh KV per window.
- **Multiple choice**: Winogrande_s (validation, 1,267), HellaSwag (validation, 10,042, 0-shot, *acc_norm*), ARC-Challenge (test, 1,172, 0-shot) and MMLU (validation, 1,531, 5-shot), all scored by continuation log-likelihood with greedy decoding, fresh KV per option, following lm-eval conventions (single-letter continuations; length-normalized scores for HellaSwag).
- **Format measurement paths**: FP16 and S-X8 v4.3 are evaluated with the fused runtime (kernel v3 with the PCA correction — the engine that defines the paper's quality numbers, PPL 10.2267); the Q8_0 reference is evaluated with llama.cpp (CUDA, Q8_1 activations, canonical perplexity tool). Prompts and scoring code are shared between engines (`mc_common.py`) to guarantee identical inputs.

The ARC letter-scoring method was validated explicitly (Appendix C of the extended protocol): answer distribution is balanced (ruling out positional-bias artifacts), per-letter accuracy is 82–95% across all answer letters, and the number reproduces across three independent engines (FP16 0.9172, S-X8 0.9164, Q8_0 0.9181). Full evidence is available in `verify_arc_method.py`.

4 Results

4.1 Quality

Metric	FP16	S-X8 v4.3	Q8_0 (unsloth)	Δ SX8−FP16	Δ Q8_0−FP16
PPL wikitext-2 (own runtime, PCA)	10.2090	**10.2267**	10.4540	+0.17%	+2.40%
PPL wikitext-2 (reference kernel)	10.2090	**10.2358**	10.4540	+0.26%	+2.40%
Winogrande_s	0.5746	**0.5722**	0.5746	−0.0024	0.0000
HellaSwag (0-shot, acc_norm)	0.6965	**0.6964**	0.6965	−0.0001	0.0000
ARC-Challenge (0-shot)	0.9172	**0.9164**	0.9181	−0.0008	+0.0009
MMLU (5-shot)	0.7133	**0.7074**	0.7087	−0.0059	−0.0046

Table 2: Quality on Qwen3.5-4B, RTX 5060 Ti. Multiple-choice tasks are robust to 8-bit-class quantization: all formats are within statistical noise (SE ≈0.5–1.4 pts). The quality differentiator of S-X8 is perplexity: **9× less loss than Q8_0** while being 11.6% smaller.

4.2 Size and memory

	S-X8 v4.3	Q8_0	FP16
Bits per weight (accounted)	**7.50**	8.50	16.00
Text weights file	**3.96 GB**	4.48 GB	9.3 GB
Complete model (vision+MTP included)	**4.38 GB (one file)**	4.48 + 0.67 mmproj = 5.15 GB	—
VRAM, text weights	**3.955 GB**	4.48 GB	9.08 GB
GGUF file (llama.cpp integration)	**3.83 GiB**	4.16 GiB	—

Table 3: Size. Text: −11.6% vs. Q8_0; complete model: −15%. The S-X8 container quantizes vision and MTP towers inside one byte-aligned file; GGUF requires a separate FP16 mmproj.

5 Speed and Engine Integration

5.1 Own runtime

- **Decode kernel** (M=1, 249 text layers, layout v4.4: 30 B/block, kb-major, coalesced warps, split-K): **80.4 tok/s** pure kernel (317 GB/s, 338 G weights/s), 7.2× over the previous generation.
- **Prompt kernel** (wmma m16n16k16, fused dequant+GEMM): **1,411 tok/s**, 4.5× faster than FP16 cuBLAS (309 tok/s) because S-X8 reads 0.94 B/weight vs. 2 B/weight.
- **CUDA Graphs** decode: **58.7 tok/s** exact (deterministic reduction, streams fixed).

5.2 llama.cpp fork (native type GGML_TYPE_SX8 = 41)

S-X8 was integrated as a first-class quantized type in a llama.cpp fork: CPU path (dequantize, vector-dot with Q8_1 activations), a CUDA **MMVQ** decode kernel, and a CUDA **MMQ** tensor-core matrix-multiply kernel (fp16 m16n8k16) so that long prompts no longer crash. Correctness was validated with isolated kernel tests (MMQ agreement with an exact CPU reference: RMS 8.4×10^{-4} on K=256 tiles), end-to-end generation, and perplexity.

llama.cpp (RTX 5060 Ti, Qwen3.5-4B)	S-X8	Q8_0
Decode tg64 (the fork, honest)	**63.79 tok/s**	—
Decode, real-world reference (community, 5060 Ti)	**63.79 tok/s**	40–54 tok/s (max 54)
Decode, local micro-benchmark (llama-bench, short context)	63.79 tok/s	72.06 tok/s
Prompt pp128 (MMQ)	1,877.85 tok/s	3,655.61 tok/s
PPL wikitext-2 in llama.cpp (no PCA path)	10.5043	10.4540

Table 4: llama.cpp integration. S-X8 decodes faster than Q8_0 in real-world use (fewer bytes per weight); in the micro-benchmark Q8_0 measures higher (72.1) but that figure excludes serving overhead. Q8_0 retains an advantage on prompt throughput because its MMQ uses years-optimized INT8 tensor cores, while the MMQ is fp16. PPL without the PCA path is equal within noise; the paper's quality number (10.2267) is defined by the PCA-enabled runtime.

[1] Real-world Q8_0 reference for the RTX 5060 Ti is reported by the community at 40–54 tok/s (consultor365 and others); the local llama-bench value (72.06) is a short-context micro-benchmark without serving overhead and is reported as such. Theoretical bandwidth ceiling ≈100 tok/s at 448 GB/s.

6 Related Work

GGUF k-quants (llama.cpp) established block-wise quantization with separate scale/zero-point handling; Q8_0 is the widely used 8-bit reference. IQ-quants push bit-rate below 4 bpp with importance-aware codebooks. AWQ and its successors select sensitive channels/layers by calibration. Hardware formats (NVFP4, MXFP4) quantize with tensor-core

support but are tied to datacenter GPUs; notably, Blackwell consumer silicon (SM120) does not expose FP4/FP8 matmul paths to cuBLAS/PyTorch, which motivated a portable ALU-only decoder. S-Quant (ICML 2026) showed that per-sub-block adaptive ranges improve quality-per-bit, a direction extended here with an exact accounting of bytes, PCA correction at the output, and full engine integration. Quantization is occasionally observed to act as a regularizer (perplexity below FP16 on small models); in the full-corpus measurements on Qwen3.5-4B the effect is absent and no claim is made of it.

7 Discussion and Limitations

- **What S-X8 wins:** perplexity (9× less loss than Q8_0), size (−11.6% text, −15% complete), VRAM (−12%), and real-world decode speed (fewer bytes per weight).
- **What it does not win:** multiple-choice accuracy is statistically identical across FP16, S-X8 and Q8_0 — these tasks are robust to 8-bit-class quantization and parity is reported honestly. Prompt throughput in llama.cpp is lower than Q8_0 (1,878 vs. 3,656 tok/s) because the MMQ kernel is fp16, not INT8; the wmma prompt kernel (1,411 tok/s) is used for the runtime path.
- **Ecosystem:** Q8_0/GGUF has universal support; S-X8 requires the runtime or the llama.cpp fork. The format spec and kernels are released under Apache-2.0 so adoption is unencumbered.
- **Scope:** results are for Qwen3.5-4B on a single GPU; generalization to other models/GPUs is expected (the decoder is architecture-independent) but not yet demonstrated end-to-end elsewhere.

8 Reproducibility

All scripts, kernels, configuration and raw JSON results are released with this paper (Apache-2.0): quantization (`quantize_qwen35_4b_sx8_v43_gpu.py`), containers (`sx8_container_v43.py`, byte-exact round-trip 381/381 tensors, PPL from file identical to in-memory), evaluation (`ppl_wikitext.py`, `ppl_fused_v44.py`, `mc_eval_sx.py`, `mc_eval_gguf.py`, shared prompts in `mc_common.py`), ARC-method validation (`verify_arc_method.py`), kernels (`cuda/sx8_fused_wmma.cu`, `cuda/sx8_decode1_v3.cu`) and the full llama.cpp fork diff. Environment: PyTorch 2.11, CUDA 13.0, numba 0.65, llama-cpp-python 0.3.34 (CUDA build), datasets 4.8.5, offline mode (HF_DATASETS_OFFLINE=1). The complete protocol, per-sample data and additional tables are in `PROTOCOLO.md`.

Appendix A The Origin of the Idea: The Shroud of Turin Studies

A.1 The study and its publication

The conceptual seeds of S-X8 come from an independent mathematical analysis of the image of the Shroud of Turin (2025–2026). All the material of that study — tests, results, re-verifications, visualizations and master documents — is published in a separate folder, complete and available, so that the full source of these ideas is at hand; a follow-up study will develop it in depth.

A.2 Nature of the work

The study was an inquiry that found very interesting things; some seem certain and astonishing, but the author, being completely honest, cannot corroborate them, nor does he intend to: that verification belongs to those with direct access to the Shroud. What does seem strongly is that a series of tests and analyses have come out veridical, and their results — and what they would mean — are astonishing; though not all of them have. Some experiments seem well done; others may be closer, in some way, to pareidolia.

A.3 Purpose of this mention

That whole exercise — whether it produced veracious results or not — served to extrapolate concepts and ideas for the format. The inquiry, the exercises and the curiosity about the Shroud of Turin allowed those ideas about S-X8 and other lines to arise. The S-X8 format, on the other hand, **has been verified** (perplexity and benchmarks in this document). The Shroud is only mentioned to show, in large part, where the extrapolation of ideas came from; its veracity or not is something the author cannot verify, but it certainly gives a great deal to think about.

A.4 Personal opinion of the author (explicitly as such)

In my opinion, others should investigate this topic much, much more.

A.5 Transparency

The study is made publicly available for several reasons: to try to be as transparent as possible and to make the source of these ideas available. It is considered the most correct thing to do, and so it has been done.

Appendix B Future Directions

The S-X8 family extends naturally toward lower bit-widths. Further formats and a next-generation architecture are under development at MarlaLabs. These are research directions; no results are claimed in this paper.